Trabajo Práctico 2

Into the Breach

Entrega 0 - Diseño preliminar

[TB025] Paradigmas de la Programación
Segundo cuatrimestre de 2026

Alumno	**Número de padrón**
Lautaro Hidalgo	112685
Jazmín Oliverio	111787
Victoria Codicetti	112025
Martín Blanco	112447
Sebastián Prato	111619

Índice

1. Introducción **2**

2. Alternativas de diseño consideradas **3**

3. Principios del Modelo1 **4**

3.1. Distribución de responsabilidades 4

3.2. Criterios de diseño 4

3.3. Contras y riesgos del modelo 5

3.4. Estado del modelo 6

4. Diagramas de clases **7**

4.1. Explicación del diagrama de clases 10

5. Diagramas de secuencia **11**

5.1. Explicación del diagrama de secuencia 11

1. Introducción

El presente informe reúne la documentación correspondiente a la Entrega 0 del segundo trabajo práctico de la materia Paradigmas de la Programación, cuyo objetivo consiste en desarrollar una adaptación del videojuego *Into the Breach* mediante el paradigma de Programación Orientada a Objetos.

En esta instancia se presenta el diseño preliminar del modelo de dominio, las alternativas consideradas durante su elaboración y las decisiones tomadas respecto de la distribución de responsabilidades entre los objetos.

Las mismas están sujetas a cambios debido a que este modelo sigue siendo una version preliminar.

El desarrollo de la solución se plantea siguiendo una metodología basada en Test Driven Development (TDD) y refactorización continua, priorizando la construcción incremental del sistema y la evolución progresiva de su diseño. Se verá como esto afectará las resoluciones tomadas en la sección de Alternativas de diseño.

Durante el proceso de modelado se priorizó respetar principios fundamentales de la orientación a objetos, tales como encapsulamiento, polimorfismo, delegación de responsabilidades, alta cohesión y bajo acoplamiento, prestando especial atención al principio *Tell, Don't Ask* y a la Ley de Demeter.

2. Alternativas de diseño consideradas

Ubicación de las unidades. La primera decisión de diseño que tuvimos que definir, y en parte la que más costó, fue el cómo modelar las unidades, casillas y posiciones. Inicialmente se consideró que cada `Unidad` almacenara una `Posicion` y recibiera el mensaje `moverseA(destino)`. De la misma forma, en esta linea de ideas las posiciones colaboraran entre sí mediante mensajes como `desplazarA` o `desplazarseAMi`, evitando exponer coordenadas. Sin embargo, modificar la posición no resolvía por sí solo la ocupación de las casillas: habría que sincronizar esa información con el tablero, con riesgo de inconsistencias.

Notamos que ibamos a tener información duplicada con este modelo, pues casilla necesitaba saber su posición, pero a su vez si casilla tiene una unidad y la unidad también conoce su posición, entonces estariamos teniendo que tener punteros de unidad a casilla y de casilla a unidad para realizar los traspases, desplazamientos y movimientos. Al final definimos que podiamos delegar la responsabilidad de movimiento de unidades a tablero.

Se optó por que cada `Casilla` conozca a su ocupante y que `Tablero` coordine los traslados. Ni `Unidad` ni `Casilla` almacenan su posición. El tablero asocia posiciones inmutables con casillas mediante `Map<Posicion, Casilla>`; `Posicion` proporciona operaciones como `obtenerAdyacente`, distancia e igualdad por valor, sin exponer sus atributos mediante getters, sino con comportamiento interno de la posición.

Recepción de ataques y ocupación. Dado que es posible atacar una casilla vacía, el ataque debe dirigirse a la `Casilla`, no directamente a un `Vek` o a un edificio. Surgió entonces el problema de enviar mensajes a un ocupante inexistente. ¿que pasa si unidad no tiene una unidad? ¿es null? estaria mandandole un mensaje a null y tendria un null pointer exception. Entonces, eso seria si la casilla esta ocupada. ¿Como lo resolvemos?

Se evaluaron tres soluciones: un condicional encapsulado en `Casilla`; subclases de casilla ocupada y desocupada; y un objeto de comportamiento intercambiable, definido mediante una interfaz de recepción de daño. Se decidió que la manera mas limpia de llevarlo a cabo seria con interfaces, sin embargo para priorizar el desarrollo TDD, al ser un modelo preliminar, hemos dejado de momento los condicionales booleanos para gestionar el estado interno de las casillas y tomar decisiones con base en eso. Si vemos que las exigencias del TP más adelante nos dan suficientes motivos, podremos implementar interfaces y sistemas más complejos.

Ataques, empujes y colisiones. Se discutió si la artillería, su ataque, las unidades o las casillas debían calcular y ejecutar los desplazamientos. Hacer que cada casilla conociera sus vecinas o que cada unidad actualizara su propia ubicación aumentaba las dependencias.

Se concluyó que `Mech` inicia el ataque, `Arma` determina sus efectos y `Tablero` coordina los empujes espaciales.

Para un empuje, el tablero obtiene las posiciones adyacentes y solicita a la casilla de origen que transfiera su unidad hacia la de destino. La casilla receptora delega en el terreno la resolución del ingreso: puede concretarse en traslado o producirse una colisión, daño y eventualmente la transformación de un edificio en suelo.

Recepción de unidades y abstracciones futuras. También se consideraron interfaces o estrategias separadas para recibir unidades cuando una casilla estuviera libre u ocupada, así como para distinguir terrenos y resolver colisiones.

Si bien estas alternativas podrían sustituir condicionales mediante polimorfismo, como se mencionó previamente, se descartaron *por ahora* las interfaces de recepción de daño y de unidades para no sobrecomplejizar el modelo inicial y priorizar el desarrollo guiado por pruebas (TDD).

Se conservaron las especializaciones de `Terreno` y `Arma` previstas en el dominio. La incorporación de nuevas abstracciones quedará sujeta a problemas concretos detectados al implementar y refactorizar.

3. Principios del Modelo1

3.1. Distribución de responsabilidades

En este modelo, las responsabilidades quedan distribuidas de la siguiente manera:

- **Tablero:** coordina los movimientos voluntarios y forzados de las unidades, gestionando las interacciones entre las casillas y las restricciones espaciales del tablero.
- **Tablero:** administra las asociaciones `Map<Posicion, Casilla>`, utilizando objetos `Posicion` como claves para localizar las casillas sin necesidad de conocer la representación interna de sus coordenadas. El alcance de movimiento se establece inicialmente como una constante compartida de valor 3.
- **Posicion:** es un objeto de valor inmutable que encapsula sus coordenadas y ofrece comportamientos como obtener una posición adyacente en determinada dirección, calcular distancias y comparar posiciones mediante igualdad por valor. No expone getters de sus coordenadas.
- **Casilla:** administra su ocupante y el terreno que contiene, sin conocer su propia posición. Recibe ataques, coordina la recepción de unidades y delega en su terreno la resolución de los efectos correspondientes.
- **Terreno:** encapsula las restricciones de ingreso, las consecuencias de los desplazamientos forzados y los efectos particulares de cada tipo de terreno. Se utiliza composición para permitir que una casilla cambie de terreno sin modificar su identidad.
- **Unidad:** encapsula sus características, estado de salud y capacidades de desplazamiento. No conoce su posición ni administra directamente los traslados entre casillas.
- **Mech:** especializa a `Unidad`, administra su arma y registra las acciones realizadas durante su turno mediante `movimientoRealizado` y `ataqueRealizado`. Inicia los ataques y delega su ejecución en el arma correspondiente.
- **Vek:** especializa a `Unidad` y encapsula los comportamientos propios de los enemigos, como anunciar y ejecutar ataques durante las fases correspondientes.
- **Arma:** encapsula las características y el comportamiento de los ataques. Interactúa con el tablero cuando sus efectos requieren localizar casillas o producir desplazamientos forzados, sin delegarle la responsabilidad del ataque.
- **Edificio:** especializa a `Terreno`, administra su propia vida y las consecuencias de recibir ataques o colisiones. Cuando es destruido, provoca la transformación del terreno de su casilla en suelo y afecta a la red eléctrica.
- **RedElectrica:** administra los puntos de energía compartidos por los edificios y permite determinar cuándo se han agotado.
- **Partida:** coordina el desarrollo del juego, su inicialización, turnos, fases y condiciones de finalización.

3.2. Criterios de diseño

- **Ocultamiento de información:** se evita exponer la representación interna de los objetos mediante getters. Se favorecen mensajes que expresen responsabilidades y comportamientos.
- **Tell, Don't Ask:** los objetos deben delegar acciones en sus colaboradores en lugar de consultar su estado para tomar decisiones que les corresponden.

- **Distribución de responsabilidades:** se busca mantener alta cohesión y bajo acoplamiento, evitando concentrar toda la lógica del juego en `Tablero` o `Partida`.
- **Ausencia de información espacial redundante:** ni `Unidad` ni `Casilla` almacenan posiciones. El tablero administra las asociaciones espaciales.
- **Composición sobre herencia cuando corresponde:** las casillas contienen terrenos que pueden reemplazarse dinámicamente. La herencia se reserva para especializaciones de comportamiento.
- **Polimorfismo:** se favorece la delegación a tipos especializados para resolver comportamientos que dependen del terreno, la unidad o el arma.
- **TDD y diseño incremental:** se prioriza implementar las funcionalidades necesarias para cada caso de uso, evitando abstracciones prematuras. Las interfaces y generalizaciones adicionales quedan sujetas a necesidades concretas de refactorización.

3.3. Contras y riesgos del modelo

1. **Centralización de responsabilidades espaciales:** `Tablero` concentra la coordinación de movimientos, empujones y consultas espaciales. Existe el riesgo de que acumule demasiadas responsabilidades conforme aumente la complejidad del juego.
2. **Dependencia del tablero para conocer ubicaciones:** como las unidades y casillas desconocen sus posiciones, cualquier comportamiento que requiera información espacial necesita colaborar con `Tablero` o recibir las posiciones correspondientes mediante mensajes.
3. **Mayor complejidad para localizar unidades:** dado que las posiciones están asociadas a casillas y no directamente a unidades, determinar la ubicación de una unidad a partir de su referencia puede requerir recorrer las asociaciones del tablero, salvo que su posición ya sea conocida por el contexto de la operación.
4. **Riesgo de vulnerar Tell, Don't Ask:** métodos como `obtenerCasilla()`, `estaLibre()`, `puedeRecibir()` o `puedeAtravesar()` pueden favorecer que otros objetos consulten información para tomar decisiones que deberían delegar. Su utilización debe evaluarse estrictamente durante la implementación, procurando reemplazar consultas de estado por mensajes que expresen comportamiento cuando corresponda.
5. **Complejidad en la coordinación de movimientos:** trasladar una unidad requiere mantener consistente la ocupación de las casillas de origen y destino. Esto se vuelve especialmente importante ante movimientos rechazados, empujones, colisiones y efectos del terreno.
6. **Costo adicional de representación:** utilizar `Map<Posicion, Casilla>` requiere implementar correctamente la igualdad y el hashing de las posiciones, además de administrar explícitamente las posiciones válidas del tablero. Como contrapartida, permite evitar el acceso externo a las coordenadas internas de `Posicion`.
7. **Acoplamiento entre los efectos del terreno y las casillas:** determinados comportamientos, como la destrucción de un edificio, requieren que el terreno provoque modificaciones en la casilla que lo contiene. Debe cuidarse que esta colaboración no genere dependencias circulares innecesarias ni responsabilidades mal distribuidas.
8. **Complejidad de las interacciones de combate:** los ataques pueden involucrar simultáneamente armas, unidades, casillas, terrenos y tablero. Existe el riesgo de generar cadenas de mensajes extensas o responsabilidades poco claras, por lo que será necesario comprobar su coherencia mediante los diagramas de secuencia y los tests.

3.4. Estado del modelo

El Modelo1 constituye una propuesta inicial de distribución de responsabilidades, construida a partir de los casos de uso analizados.

Las decisiones actuales sirven como base para comenzar la implementación mediante TDD. No se consideran definitivas aquellas abstracciones cuya necesidad todavía no haya sido demostrada por los casos de uso.

Los futuros cambios deberán justificarse mediante problemas concretos de diseño, nuevas reglas del dominio o necesidades detectadas durante las pruebas y refactorizaciones.

4. Diagramas de clases

El siguiente diagrama de clases UML representa la estructura estática del Modelo1, las relaciones entre los distintos objetos del dominio y las responsabilidades inicialmente asignadas a cada uno.

En particular, se representan las asociaciones entre `Tablero`, `Casilla` y `Posicion`, las jerarquías de unidades, armas y terrenos, y los objetos responsables de administrar el desarrollo de la partida.

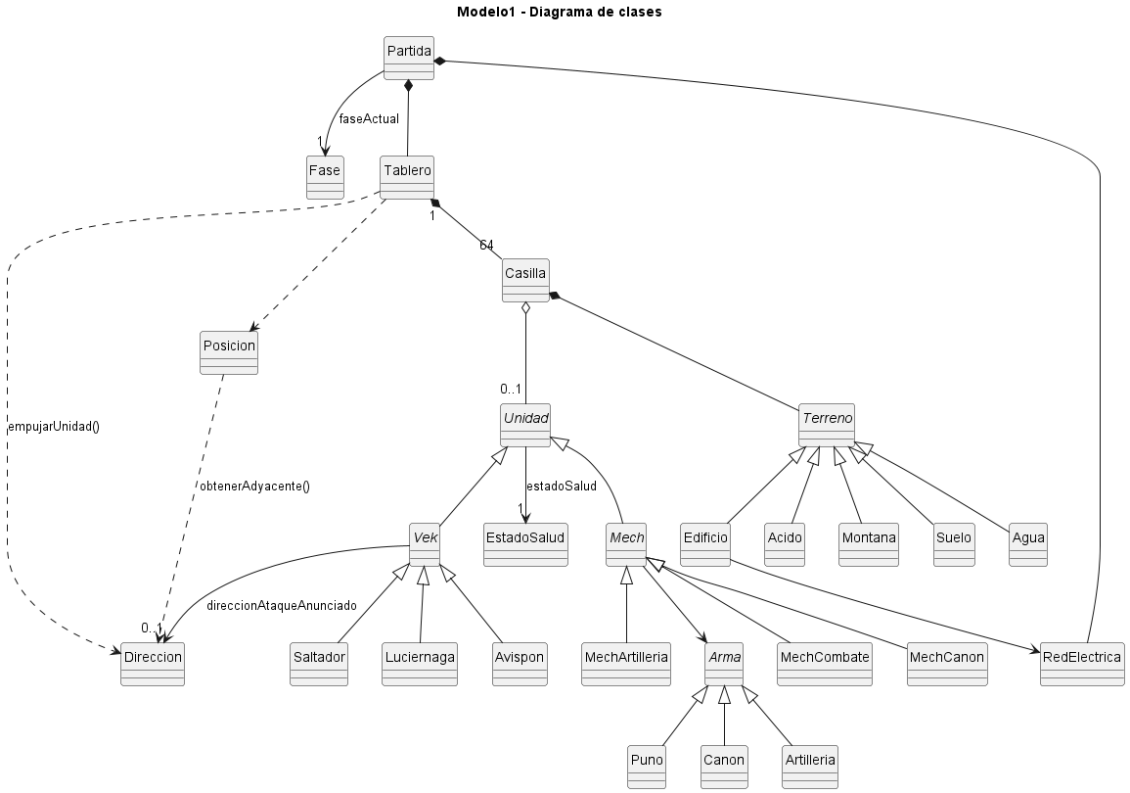


Figura 1: Diagrama de clases UML del Modelo1.

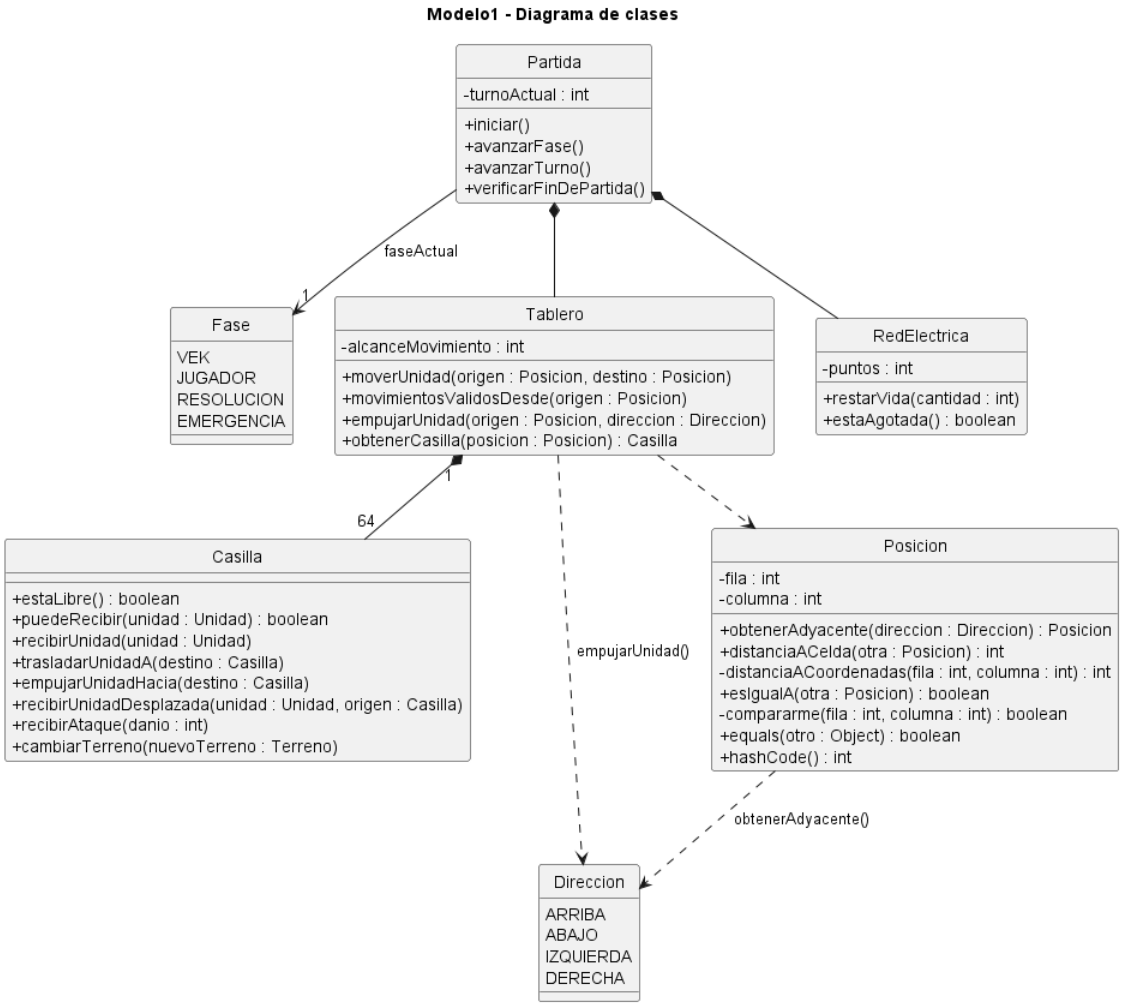


Figura 2: Diagrama de clases UML de Partida.

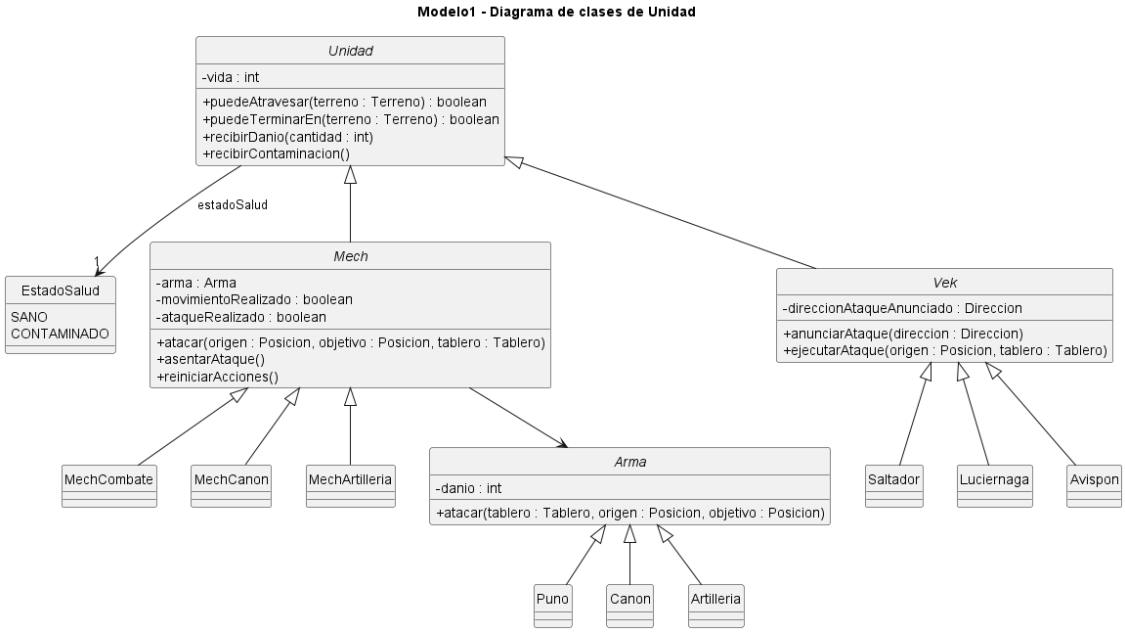


Figura 3: Diagrama de clases UML de Unidad.

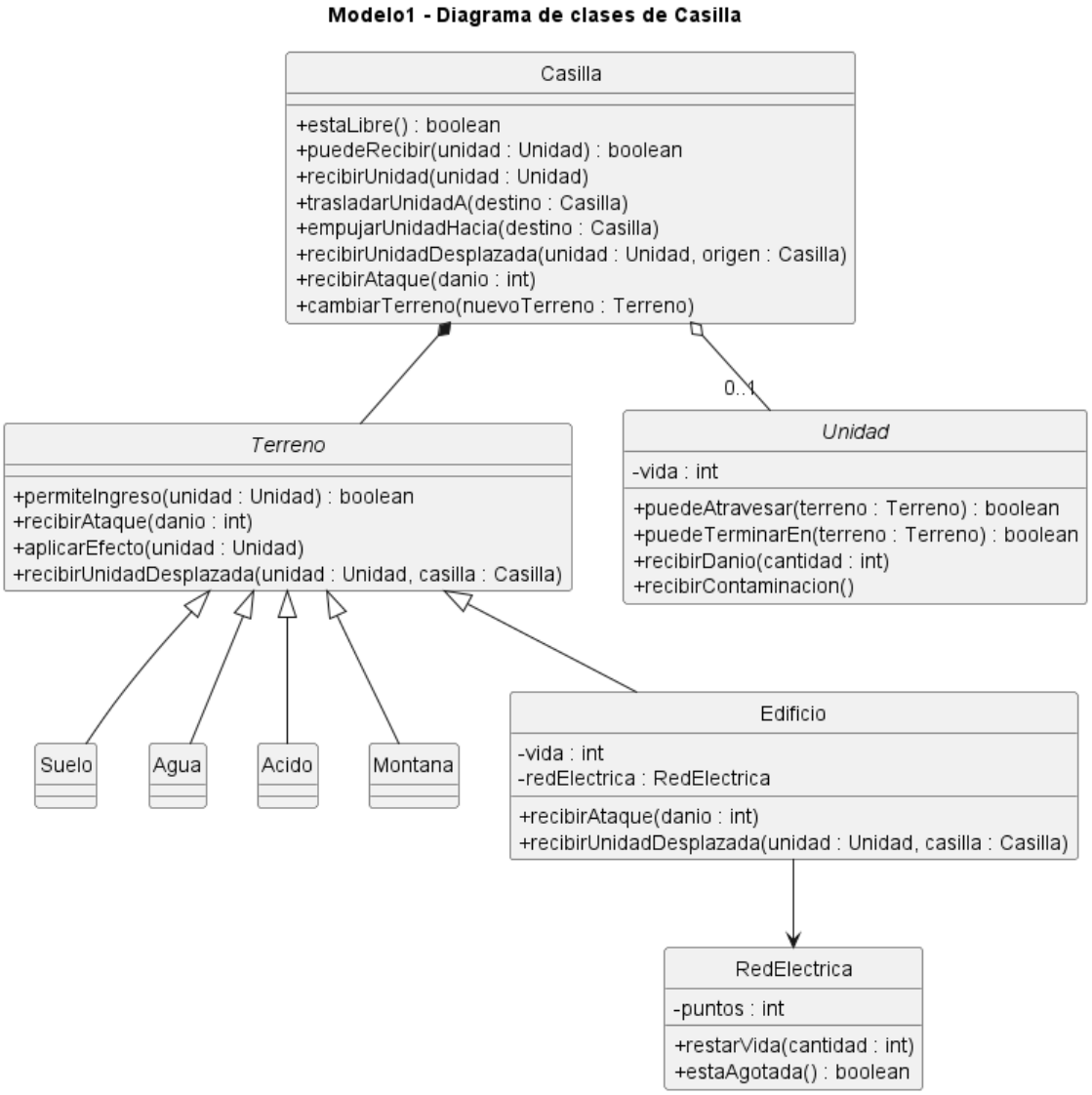


Figura 4: Diagrama de clases UML de Casilla.

4.1. Explicación del diagrama de clases

El sistema se organiza alrededor de la clase `Partida`, encargada de coordinar el desarrollo del juego y administrar las referencias al tablero y a la red eléctrica. `Tablero` mantiene una asociación entre objetos `Posicion` y `Casilla`, permitiendo localizar las distintas casillas y coordinar los movimientos de las unidades. Cada `Casilla` contiene un objeto `Terreno` y puede estar ocupada por una `Unidad`. Los distintos terrenos especializan su comportamiento mediante una jerarquía que contempla suelo, agua, ácido, montaña y edificio. Las unidades se especializan en `Mech` y `Vek`. Los Mechs poseen armas que determinan el comportamiento de sus ataques, mientras que los Veks encapsulan sus propias acciones ofensivas. La composición entre casillas y terrenos permite modificar dinámicamente el terreno asociado a una casilla sin necesidad de reemplazar la propia casilla.

5. Diagramas de secuencia

Los diagramas de secuencia representan la colaboración entre los distintos objetos durante la ejecución de casos de uso relevantes del modelo.

Para esta entrega se presenta el escenario en el cual un Mech de Artillería ataca una casilla ocupada por un Vek y provoca el desplazamiento forzado de otra unidad contra un edificio.

El caso permite ilustrar la distribución de responsabilidades entre las unidades, las armas, el tablero, las casillas y los terrenos, así como las consecuencias de una colisión sobre la red eléctrica.

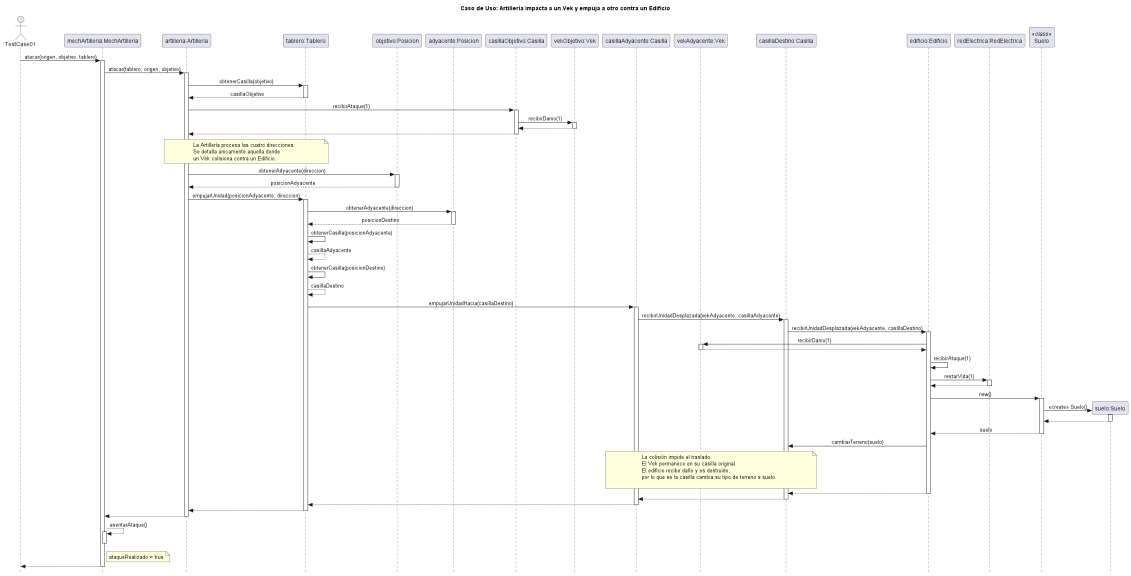


Figura 5: Diagrama de secuencia: ataque de Artillería, empuje y colisión contra un edificio.

5.1. Explicación del diagrama de secuencia

El caso de uso comienza cuando un objeto `MechArtilleria` recibe el mensaje para atacar una posición objetivo. El Mech delega la ejecución del ataque en su objeto `Artilleria`, que obtiene la casilla objetivo mediante el tablero y le solicita recibir el ataque correspondiente. Posteriormente, el arma determina las posiciones adyacentes a la casilla objetivo y solicita al tablero los empujes necesarios. El tablero coordina el desplazamiento forzado, mientras que las casillas involucradas delegan en sus terrenos la resolución del ingreso. En el escenario representado, el desplazamiento es impedido por un edificio, lo que provoca una colisión y la aplicación de daño a la unidad y al edificio. Como consecuencia de la destrucción del edificio, se descuentan puntos de la red eléctrica y el terreno de la casilla se transforma en suelo. Finalmente, el Mech registra la ejecución de su ataque mediante el mensaje `asentarAtaque()`, modificando su estado interno para impedir una segunda acción de ataque durante el mismo turno.